

Arquitectura del sistema — NESGESFinanceTrust

Plataforma: **nesgesfinancetrust.com** · Versión **v3.4-dev** (Agosto 2026)

Empresa: **NESGESFinance Ecosystem S.A.S. BIC. & LLC.** · EIN: 0008086872

CEO-Fundador: Cbr. Joan Santiago Ramírez Almeida

Lema: “Y a tu prójimo como a tí mismo”

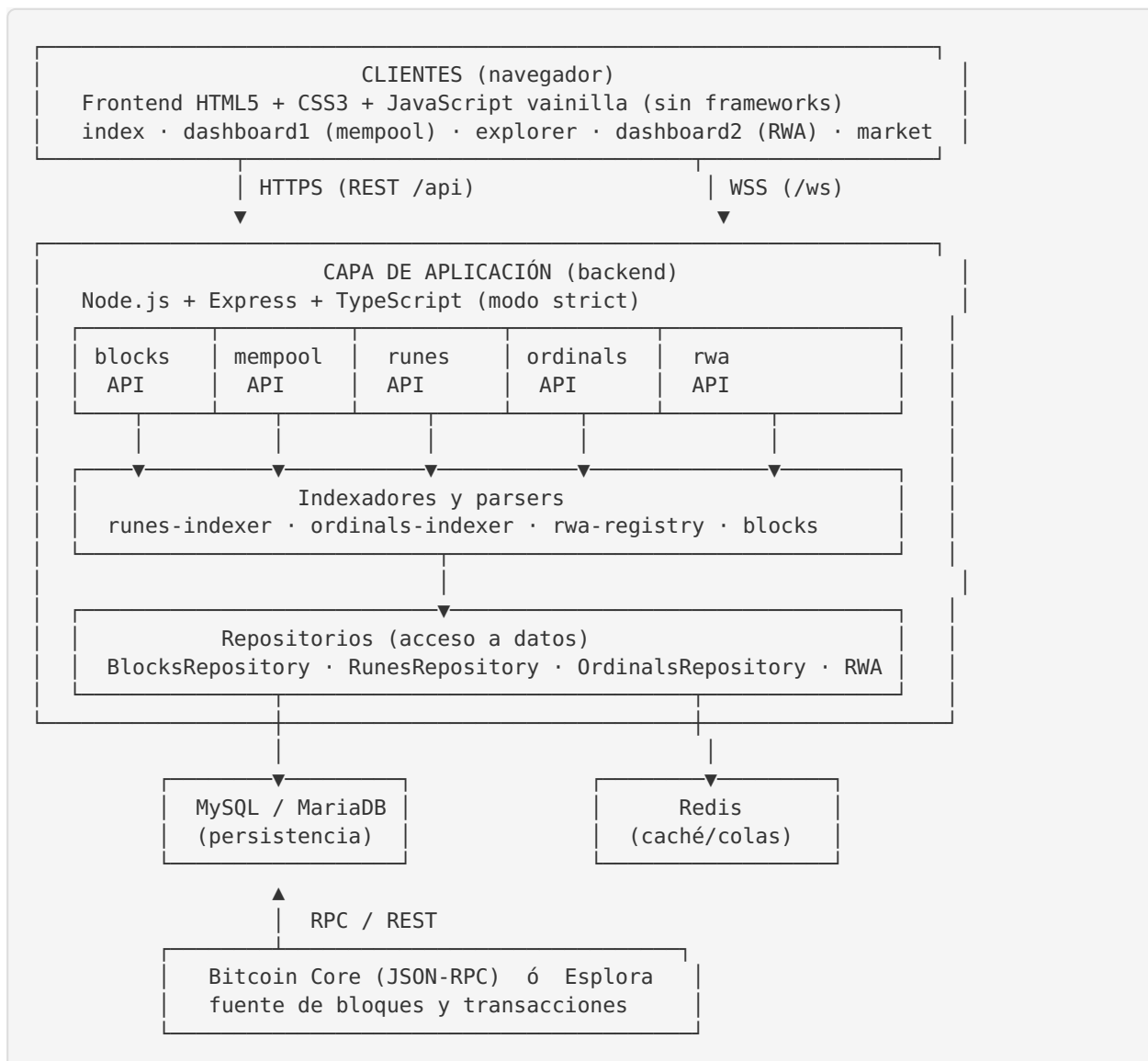
Este documento describe la arquitectura de la plataforma de indexación y tokenización de la NESGESFinance Ecosystem, construida sobre Bitcoin y sus protocolos de capa 1 y capa 2.

1. Visión general

NESGESFinanceTrust indexa la cadena de Bitcoin y expone una capa de servicios para tres clases de activos digitales:

- **Runes** → Utility Tokens (fungibles, protocolo de Casey Rodarmor).
 - **Ordinals** → Security Tokens y contenedores de metadatos de RWA.
 - **Taproot Assets + Lightning** → liquidez y liquidación en capa 2 (L2).
-

2. Diagrama de capas



3. Flujo de datos

- Ingesta.** `bitcoin-api-factory` selecciona el backend de datos: `bitcoin-client` (JSON-RPC de Bitcoin Core) o `esplora-client` (REST).
- Indexación de bloques.** `blocks.ts` recupera cada bloque nuevo, calcula estadísticas (comisiones, tamaño, CPFP, ajuste de dificultad) y las persiste mediante `BlocksRepository`.
- Parseo de protocolos.**
 - `runes-parser` decodifica los RuneStone del `OP_RETURN` (`OP_13`, `LEB128`).
 - `inscription-parser` extrae las inscripciones del witness Taproot.
- Tokenización RWA.** `rwa-registry` vincula un Ordinal (titularidad legal) y, opcionalmente, un Rune (fraccionamiento) a un activo del mundo real, tras pasar `rwa-validator` (KYC/AML/MiCA).
- Persistencia.** Los repositorios escriben en MySQL/MariaDB; Redis cachea consultas calientes y sirve de cola para el difusor WebSocket.

6. **Difusión.** `websocket-handler` publica los eventos por canal (`blocks`, `mempool`, `runes`, `ordinals`, `rwa`) a los clientes suscritos.
7. **Presentación.** El frontend consume REST para carga inicial y WebSocket para actualizaciones en vivo.

4. Componentes del backend

Módulo	Responsabilidad
<code>index.ts</code>	Arranque del servidor Express y montaje de rutas.
<code>config.ts</code>	Configuración tipada (incluye sección <code>MEMPPOOL</code>).
<code>logger.ts</code>	Registro estructurado (Winston).
<code>database.ts</code>	Pool de conexiones MySQL/MariaDB.
<code>api/blocks.ts</code>	Indexación y estadísticas de bloques.
<code>api/mempool.ts</code>	Estado del mempool y estimación de comisiones.
<code>api/runes/*</code>	Parser, indexador y API de Runes.
<code>api/ordinals/*</code>	Parser, indexador y API de Ordinals.
<code>api/rwa/*</code>	Validador, registro y API de RWA.
<code>bitcoin/*</code>	Clientes RPC/Esplora y utilidades criptográficas.
<code>repositories/*</code>	Acceso a datos por dominio.
<code>interfaces/*</code>	Contratos de tipos compartidos.
<code>utils/*</code>	<code>bitcoin-script</code> , <code>varint</code> , <code>compliance</code> .

5. Seguridad

- **TypeScript strict** con `noUnusedLocals` / `noUnusedParameters` para reducir errores en tiempo de compilación.
- **Validación de entrada** en todos los endpoints (parámetros y cuerpos).
- **Cumplimiento normativo** (KYC/AML/MiCA) obligatorio antes de tokenizar RWA.
- **Integridad documental** mediante hash SHA-256 de cada documento legal.

- **Separación de secretos** vía variables de entorno (`.env` , nunca en el repo).
 - **Cabeceras CORS** controladas y límites de tamaño de cuerpo (`2mb`).
-

6. Escalabilidad

- **Caché Redis** para consultas frecuentes y para desacoplar la difusión.
 - **Indexación incremental** por altura de bloque (reanudable).
 - **Repositorios desacoplados** que permiten fragmentación o réplica de lectura.
 - **Frontend estático** servible desde CDN o Nginx, independiente del backend.
 - **Contenedores Docker** (ver `docker-compose.yml`) para despliegue horizontal.
-